

Credit Risk Scoring System

Project Report

A binary classification project that estimates the probability that a loan applicant will default. The repository contains the analysis and model-development notebook, the credit dataset, a saved CatBoost-based model and decision threshold, a FastAPI prediction service, and a browser-based application.

Live application: [Credit Risk Analyzer](#)

Project contents

Path	Purpose
trials.ipynb	Exploratory analysis, data cleaning, model comparison, tuning, threshold selection, calibration, SHAP explanations, and artifact export.
dataset/credit_dataset.csv	Applicant and loan records with loan_status as the binary target.
CatBOost.pkl	Serialized calibrated CatBoost pipeline used by the API (the filename's capitalization is intentional).
catThresold.pkl	Serialized probability threshold used by the API (the filename spelling is intentional).
main.py	FastAPI application, input schema, preprocessing, inference, and static-file hosting.
static/	Browser interface: index.html, style.css, and script.js.
requirements.txt	Python libraries for analysis, machine learning, and serving.
runtime.txt	Python runtime declaration (Python 3.12.x).
render.yaml	Render web-service build and start configuration.
.env.example	Example model path settings; the current application loads artifacts by fixed filenames next to main.py.

Project structure

```
Credit Risk Scoring System/
├── dataset/
│   └── credit_dataset.csv    # Applicant data used for training and analysis
├── static/
│   ├── index.html          # Web interface
│   ├── script.js           # Form handling and API requests
│   └── style.css            # Interface styles
├── trials.ipynb            # Exploration, training, evaluation, and artifact creation
├── main.py                 # FastAPI application and prediction endpoint
├── CatBOost.pkl            # Calibrated model pipeline loaded by the API
├── catThresold.pkl         # Decision threshold loaded by the API
├── requirements.txt        # Python dependencies
├── runtime.txt             # Python runtime version for deployment
├── render.yaml            # Render service configuration
├── .env.example            # Example environment settings
├── .gitignore              # Files excluded from version control
├── LICENSE                 # Project license
└── README.md               # Project documentation
```

Dataset and prediction fields

The notebook's initial dataset inspection reports 32,581 records and 12 columns. The target is `loan_status` (0 or 1). The 11 input features used by the model/API are:

Feature	Meaning / expected type
<code>person_age</code>	Applicant age in years (integer).
<code>person_income</code>	Annual income (numeric).
<code>person_home_ownership</code>	Home ownership category (string).
<code>person_emp_length</code>	Employment length in years (numeric).
<code>loan_intent</code>	Loan purpose category (string).
<code>loan_grade</code>	Loan grade (string).
<code>loan_amnt</code>	Loan amount (numeric).
<code>loan_int_rate</code>	Interest rate (numeric).
<code>loan_percent_income</code>	Loan amount as a share of income (numeric).
<code>cb_person_default_on_file</code>	Prior default-on-file indicator (string, such as Y/N).
<code>cb_person_cred_hist_length</code>	Credit history length in years (integer).

The initial notebook inspection showed missing values in `person_emp_length` (895 records) and `loan_int_rate` (3,116 records). Numeric and categorical imputers are included in the modeling pipelines so missing feature values can be handled during fitting. The dataset's categorical fields are `person_home_ownership`, `loan_intent`, `loan_grade`, and `cb_person_default_on_file`.

Workflow followed in the notebook

The steps below reflect the work recorded in `trials.ipynb`.

1. **Import the analysis and modeling stack.** The notebook imports NumPy, pandas, Matplotlib, Seaborn, scikit-learn preprocessing, model selection and metrics, Logistic Regression, Random Forest, CatBoost, XGBoost, SciKeras/TensorFlow, SHAP, and Joblib. Warnings are suppressed for notebook output.
2. **Load and inspect the data.** Read `dataset/credit_dataset.csv` into `df_org`; inspect sample rows, dimensions, column names, types, null counts, categorical values, and target labels/class counts. Plot the target balance.
3. **Explore feature distributions.** Identify numeric predictors; plot their boxplots and descriptive statistics; count unusually high ages (>100) and employment lengths (>60); and visualize correlations among numeric fields.
4. **Create a working copy and clean records.** Copy the original frame to `df`, remove duplicate rows, retain ages from 18 through 100, require employment length not to exceed age and not to exceed 60 years, and retain only positive loan amounts. The cleaned frame reported by the notebook has 31,522 rows. Missing values are not dropped; model pipelines impute them.
5. **Separate predictors and target.** Set `X` to all fields except `loan_status` and `y` to `loan_status`; identify numeric and categorical predictors.

6. **Create train and test sets.** Use an 80/20 split with `random_state=42` and stratification on `y`. Compute the negative-to-positive training class ratio for XGBoost's `scale_pos_weight`.
7. **Build model-specific preprocessing.** Numeric values use median imputation. Logistic Regression and the neural network also standardize numeric values; all five model paths impute categories with the constant N/A and one-hot encode them with unknown categories ignored. The notebook also defines this preprocessing for CatBoost, so the stored estimator expects the transformed numeric feature matrix.
8. **Compare five candidate model families.** Configure stratified five-fold cross-validation (`shuffle=True`, `random_state=42`) and report ROC AUC, accuracy, precision, recall, and F1 for:
 - Balanced Logistic Regression (`max_iter=1000` in the CV pipeline).
 - Balanced Random Forest (500 trees in the CV pipeline).
 - XGBoost (weighted for the class imbalance).
 - CatBoost (500 iterations, learning rate 0.1, balanced class weights).
 - A feed-forward neural network with 64 and 32 ReLU units, dropout, and a sigmoid output; it uses Adam and binary cross-entropy.
1. **Fit the candidates and evaluate classification behavior.** The notebook fits candidate pipelines, then defines an evaluator that prints accuracy, precision, recall, F1, confusion matrix, and classification report, and draws confusion-matrix and precision-recall plots. Some early evaluator calls pass `X_train`, `y_train` (notably Random Forest, CatBoost, and the neural network); those outputs are training-set metrics and should not be read as held-out test performance. Later tuned-model checks use the held-out test set.
2. **Tune three tree models by randomized search.** Use `RandomizedSearchCV` with the same stratified five-fold split and `average_precision` scoring. Search XGBoost over 150 parameter samples; search Random Forest and CatBoost over 10 samples each. The notebook records best cross-validation average-precision scores of about 0.90 for XGBoost, 0.88 for Random Forest, and 0.90 for CatBoost. The recorded CatBoost parameters include 929 iterations, depth 5, learning rate about 0.125, `l2_leaf_reg=2`, and `random_strength` about 4.63.
3. **Evaluate tuned models and select a decision threshold.** Evaluate tuned models on the test partition and inspect precision/recall tradeoffs. For CatBoost, calculate the precision-recall curve from positive-class probabilities, calculate F1 along the curve, and select the threshold with the highest F1. The recorded tuned CatBoost test ROC AUC is approximately 0.951. The selected numeric threshold is stored in `catThresold.pkl`.
4. **Calibrate probabilities.** Wrap the best tuned CatBoost pipeline with `CalibratedClassifierCV(method='isotonic', cv=5)`, fit it on the training data, and plot calibration curves for calibrated and uncalibrated predictions on the test data.
5. **Explain predictions with SHAP.** Transform test examples with the best CatBoost pipeline, create a `TreeExplainer`, calculate SHAP values, and generate a global summary plot plus a waterfall explanation for one test example.
6. **Export artifacts.** The notebook saves the calibrated classifier as `CatBOost.pkl` and the F1-selected threshold as `catThresold.pkl` using Joblib.

Cross-validation results recorded in the notebook

These are the mean scores printed for the initial five-fold model comparison, before tuning. They are cross-validation scores on the training partition, not final test-set scores.

Model	ROC AUC	Accuracy	Precision	Recall	F1
Logistic Regression	0.871	0.812	0.545	0.778	0.641
Random Forest	0.929	0.927	0.903	0.742	0.815
XGBoost	0.939	0.909	0.791	0.786	0.788
CatBoost	0.945	0.921	0.837	0.787	0.811
ANN	Not available	0.928	0.959	0.694	0.805

For the ANN, the recorded ROC AUC was nan, so the value is not available.

Scores can vary when the notebook is rerun if library versions, runtime, or random behavior differ. The XGBoost comparison cell spells the estimator argument `n_estimator` (singular), while its later tuning uses `n_estimators`; consult the notebook output/environment if reproducing that particular run.

Run locally

Use Python 3.12 as declared in `runtime.txt`. From the project root, create and activate a virtual environment, then install dependencies:

```
python -m venv .venv
```

On Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

On macOS/Linux:

```
source .venv/bin/activate
```

Install and start the service:

```
python -m pip install -r requirements.txt
uvicorn main:app --reload
```

The app serves its interface at `http://127.0.0.1:8000/`. FastAPI's interactive API documentation is at `http://127.0.0.1:8000/docs`. Both `CatBOost.pkl` and `catThreshold.pkl` must be present in the project root when the service starts. The artifacts are loaded during application startup.

To explore or reproduce the model-development workflow, open `trials.ipynb` in Jupyter from the project root; the notebook reads the dataset using the relative path `dataset/credit_dataset.csv`.

Prediction API

POST /predict accepts JSON with all 11 model features. For example:

```
{
  "person_age": 30,
  "person_income": 60000,
  "person_home_ownership": "RENT",
  "person_emp_length": 5,
  "loan_intent": "PERSONAL",
  "loan_grade": "B",
  "loan_amnt": 10000,
  "loan_int_rate": 11.5,
  "loan_percent_income": 0.17,
  "cb_person_default_on_file": "N",
  "cb_person_cred_hist_length": 7
}
```

The endpoint returns:

```
{
  "default_probability": 0.12,
  "default_prediction": 0,
  "threshold": 0.35,
  "result": "Low Risk",
  "confidence": 35.0
}
```

Values above are illustrative; actual probability, threshold, and confidence depend on the saved artifacts and input. `default_prediction` is 1 when the estimated positive-class probability is at least the saved threshold and 0 otherwise. `result` maps these labels to High Risk and Low Risk. The response key `threshold` preserves the spelling used in the existing API. `confidence` is currently calculated as a normalized distance from the threshold; it is not a statistically calibrated measure of prediction certainty.

The browser form gathers the same fields and sends the request to /predict. Categorical options in the form cover home ownership, loan purpose, loan grade, and prior-default indicator. The API code catches prediction exceptions and returns an error object; input-schema validation is handled by FastAPI/Pydantic.

Deployment

render.yaml defines a Render Python web service named credit-risk-scoring, installs requirements.txt, and starts uvicorn main:app --host 0.0.0.0 --port \$PORT. Ensure the model artifacts are available in the deployed project directory at startup. runtime.txt specifies Python 3.12.x.

The deployed application is available at <https://credit-risk-analyzer-ml-system.onrender.com/>.

Reproducibility and artifact notes

- The notebook saves a **calibrated** CatBoost pipeline, while the tuned threshold is selected from probabilities generated by the uncalibrated random_search_ctb model. The API applies the saved threshold to probabilities from the calibrated artifact, so that threshold/probability pairing differs from the threshold-selection step in the notebook.
- The notebook's variable is named best_threshold_cat; the artifact filename and API dictionary key also retain the existing threshold typo.
- main.py loads CatBoost.pkl and catThreshold.pkl relative to its own file location. The .env.example names different files (CatBoost.pkl, catThreshold.pkl), and main.py does not currently read those environment variables.
- joblib artifacts can depend on compatible Python and ML-library versions. For reproducibility, preserve the environment used to train and serialize them alongside the artifacts.
- requirements.txt lists notebook and serving dependencies without pinned versions. The model notebook includes scikit-learn, CatBoost, XGBoost, TensorFlow/SciKeras, SHAP, plotting, and data-processing libraries; the service uses FastAPI, Pydantic, pandas, NumPy, and Joblib (Joblib is available through scikit-learn dependencies, but is not separately listed).

License

See LICENSE.

System diagram

